Quell debate: roadmap

Status: **first draft, 2026-11-03, for discussion.** This document pulls the designs, plans and lab work so far into one plan, from labelled texts to a debate you can play in Quell. Where it makes a choice the existing docs haven't made, the choice is marked **Decision needed**. Blue explainer boxes cover technical terms, mostly on the NLP side.

Contents

1. Overview 2
   1.1. The goal 2
   1.2. The main idea: analyse while authoring, evaluate while playing 2
   1.3. Background in one page 3
   1.4. What exists today 4
2. The NLP pipeline 6
   2.1. Principle: rules, oracles and a reviewer 6
   2.2. Step 1: segment the text into locutions 7
   2.3. Step 2: relate the units 7
   2.4. Step 3: reconstruct canonical statements 8
   2.5. Step 4: analyse each statement 8
   2.6. Step 5: type each argument 9
   2.7. Step 6: tag concepts 9
   2.8. How the pipeline is measured 9
   2.9. How the pipeline meets the editor: suggest, then review 10
   2.10. Implementation choices 10
3. Integration into Quell 12
   3.1. The player's loop 12
   3.2. Runtime architecture 13
4. Phases 17
   4.1. Track A: content and NLP 17
   4.2. Track G: the game 18
   4.3. Deferred 18
5. Risks and open questions 18
   5.1. Glossary 19
   5.2. References 19

1. Overview

1.1. The goal

Quell's players should get real practice debating real philosophical questions: questions without easy answers. Some debates must be won to move on. In others you move on either way, and the result only changes the conversation. Players learn by **reading in-world texts** (excerpts of real philosophy), **taking notes**, and **building arguments** out of what they've found. Between runs, they keep only what they know. This is the roguelite loop, in the spirit of *Outer Wilds*.

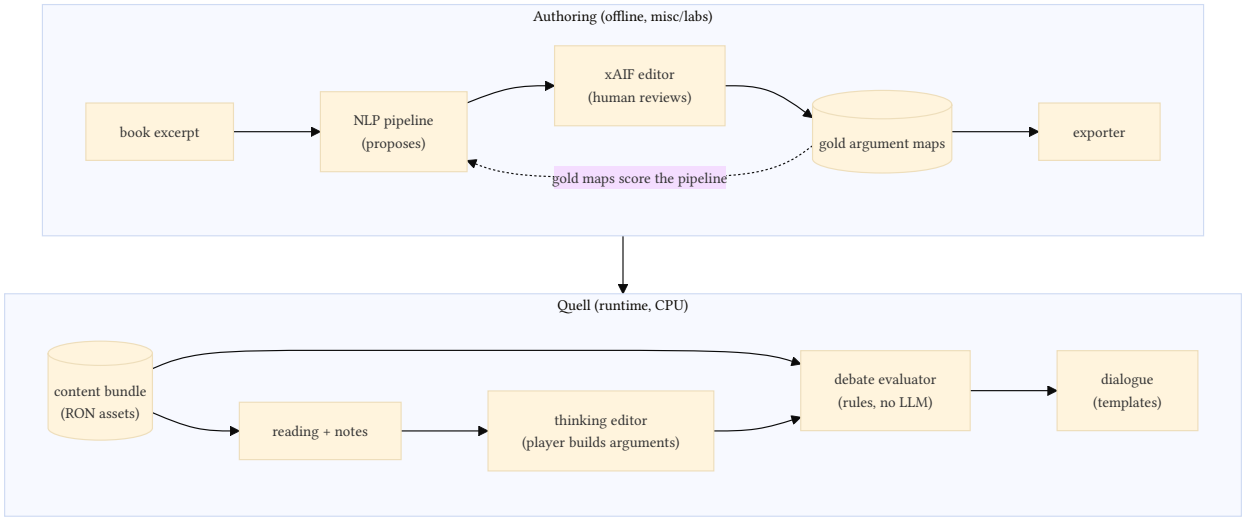
Three constraints shape every decision below:

- **CPU only.** The GPU is busy rendering. The target machine has a weak GPU and a good CPU.
- **One developer.** Anything that needs a team to maintain is out.
- **The texts are real books.** The lab never commits or publishes them. Shipping them in the game is a separate question (Section 5).

1.2. The main idea: analyse while authoring, evaluate while playing

Typing an arbitrary argument correctly is hard for people and harder for models. So the plan avoids doing it at runtime:

1. **At authoring time** (offline, in the lab), every text the player will read becomes an **argument map**. The map holds the text's statements, how they connect, and the PTA type of every argument. Models *propose* the map, and a human checks and fixes it in the xAIF editor. Here any model size is fine, and so is slow.
2. **At play time**, the player doesn't type free text. They build arguments in a **graph editor** (the "thinking" interface) out of statements they've collected. Every statement is already analysed, so the game can type the player's argument *by rule*, with no language model. It then evaluates the argument against the NPC's position.



Two consequences matter for planning:

- **The game doesn't need the NLP pipeline to exist.** Maps made entirely by hand, as we're making them now, are enough to build a debate. The pipeline only makes authoring faster. So the game's vertical slice and the pipeline can be built in parallel (Section 4).
- **The maps we're making now are the gold data.** Every reviewed map in `misc/labs/xAIF/examples` is both game content and a test case for the pipeline.

This replaces two earlier assumptions. One is the free-text player input in `capna-based-flow.md`. The other is the in-game sketch in `pta-atip-algorithm.typ` (§ "In-game use"), which typed the player's free text at runtime.

1.3. Background in one page

1.3.1. The Periodic Table of Arguments (PTA)

In the PTA, an argument is three statements:

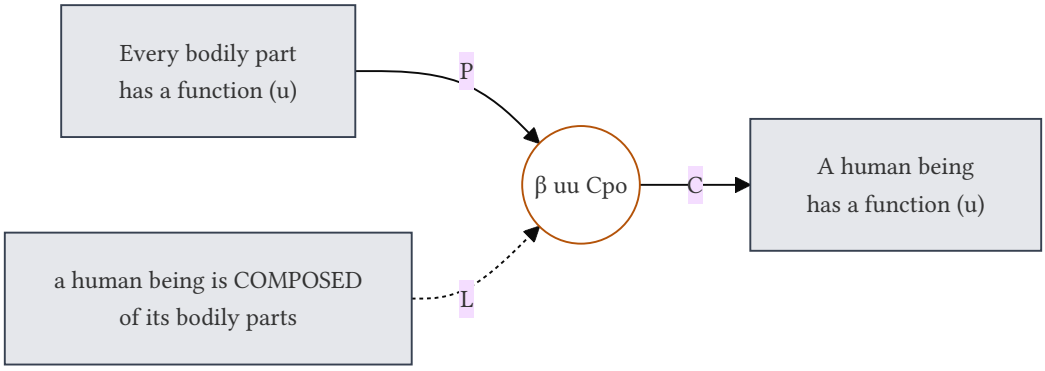
- a **conclusion**;
- a **premise**;
- a **lever**, saying *how* the premise supports the conclusion. The lever is usually unstated.

Each statement has a **subject** and a **predicate**. The argument's type follows from three things:

- **form**: what the conclusion and premise share.
  - α shares the subject: "a is X because a is Y".
  - β shares the predicate: "a is X because b is X".
  - γ shares nothing.
  - δ is about a whole proposition: "q is acceptable because q was said by Z".
- **substance**: what kind of statements or terms are involved. For α it is the statement kinds (fact, value, policy). For β it is the subjects' scope (singular, particular, universal). γ and δ have their own values.
- **lever**: the relation itself, named by a keyword (CAUSE, EXAMPLE, ANALOGOUS…).

There are 50 types, written as codes like `α FF Cau` or `β up Exa`. The *Argument Type Identification Procedure* (ATIP) finds a type in six steps: label, reformulate, form, substance, lever, name. We draw an argument as a connector k with three edges:

- a premise edge (P) into k;
- an optional lever edge (L) into k;
- a conclusion edge (C) out of k.



1.3.2. Inference Anchoring Theory (IAT) and xAIF

The PTA describes single arguments. **IAT** describes a whole text or dialogue, in two layers joined by a third:

- **locutions** (L): exactly what was said, and by whom. **Transitions** (TA) link one locution to the next.
- **propositions** (I): what was asserted, reconstructed. **Relations** link them: inference (RA), conflict (CA) and rephrase (MA).
- **illocutionary connections** (YA) anchor the second layer in the first. Asserting, Arguing and Disagreeing are YAs.

xAIF is the JSON format ARG-tech's tools use for IAT graphs. Our editor reads and writes xAIF, with two additions: PTA data on each node (a `pta` field), and a `lever` role for edges. In this model:

- ATIP's reformulation step lives on the YA (L ⟶ YA ⟶ I).
- An RA's form and substance are *derived* from its canonical statements, never typed in by hand.

The details are in the `xaif-iat` skill and the editor's README. `examples/ra-inputs.json` lists every way into an RA.

1.3.3. CAPNA: judging an argument

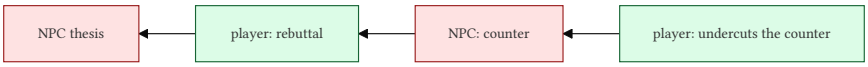
ATIP says *what kind* of argument something is, not whether it is good. Hinton and Wagemans' **CAPNA** procedure adds three judging stages:

- **process**: is it relevant to the issue?
- **reasoning**: is the premise acceptable, and does the lever hold?
- **language**: are the terms clear?

Each lever type also has its own **critical questions**. For an analogy: "is a really like b in the way that matters?" For an authority: "is the source an expert on this?" In Quell, these questions become what the NPC says back.

1.3.4. Winning a debate: argumentation frameworks

> **Explainer: abstract argumentation.** Dung (1995) treats a debate as a directed graph. Nodes are arguments, and an edge $A \to B$ means "A attacks B". Rebuttals, undercuts and underminers are all attacks. An argument is **accepted** if every attack on it is defeated by an accepted argument. The most cautious set of accepted arguments is the **grounded extension**, computed by a simple loop: accept everything unattacked, reject what those attack, and repeat. That takes polynomial time, and at game scale it costs nothing.



In this example:

1. C is unattacked, so it is accepted (green).
2. C defeats B, so B is rejected (red).
3. A's only attacker is defeated, so A is accepted.
4. An accepted argument attacks the NPC's thesis, so the thesis is rejected.

Winning means the NPC's thesis is out of the grounded extension: the player made an attack the NPC can't answer from its repertoire.

> **Explainer: value-based argumentation.** Bench-Capon (2003) adds **values**. Each argument promotes a value (liberty, order, piety…), and each **audience** ranks values. An attack only succeeds against an audience that doesn't rank the attacked argument's value higher. So the same argument can win against one NPC and fail against another, and the game never has to declare a value judgement true. Each NPC's value ranking becomes part of their authored personality.

1.4. What exists today

piece	state
xAIF editor `misc/labs/xAIF`	A Svelte app with the IAT layers, PTA derivation, typed ports, lint, sessions and span handles. It is its own local repo, with no remote. 34 tests.
Argument maps `xAIF/examples`	Reviewed: Aquinas (`aquinas-llm.json`) and Engels. Drafted and awaiting critique: Aristotle's function argument. Every way into an RA: `ra-inputs.json`.
Judgement calls `xAIF/judgement-calls.md`	Annotation conventions, each settled, proposed or open, with the example that raised it. This is the seed of the labelling guidelines.
Pilot corpus `nlp/data/pta-corpus`	31 excerpts from 29 writers (listed in `sources.tsv`), never committed. Also 10 single-argument examples from the PTA website (`pta-test`).

piece	state
note-lab `misc/labs/nlp`	Python (uv): the data format, `notelab check`, tier 0 concept coverage, the PTA schema generator for INCEpTION, and the labelling guide and methods docs.
Designs	`pta-atip-algorithm.typ` (ATIP as rules plus ten oracles), `pta-document-analysis.typ` (finding arguments in a text), `capna-based-flow.md` (judging and response categories), `quell-reading-notes-design.md` (notes → concepts, tiers 0–2).
Skills	`pta-atip`, `xaif-iat`, `inception-api`.
Quell	No debate, reading or dialogue code yet. The lab's `examples/*.yarn` files sketch the dialogue side in Yarn.

2. The NLP pipeline

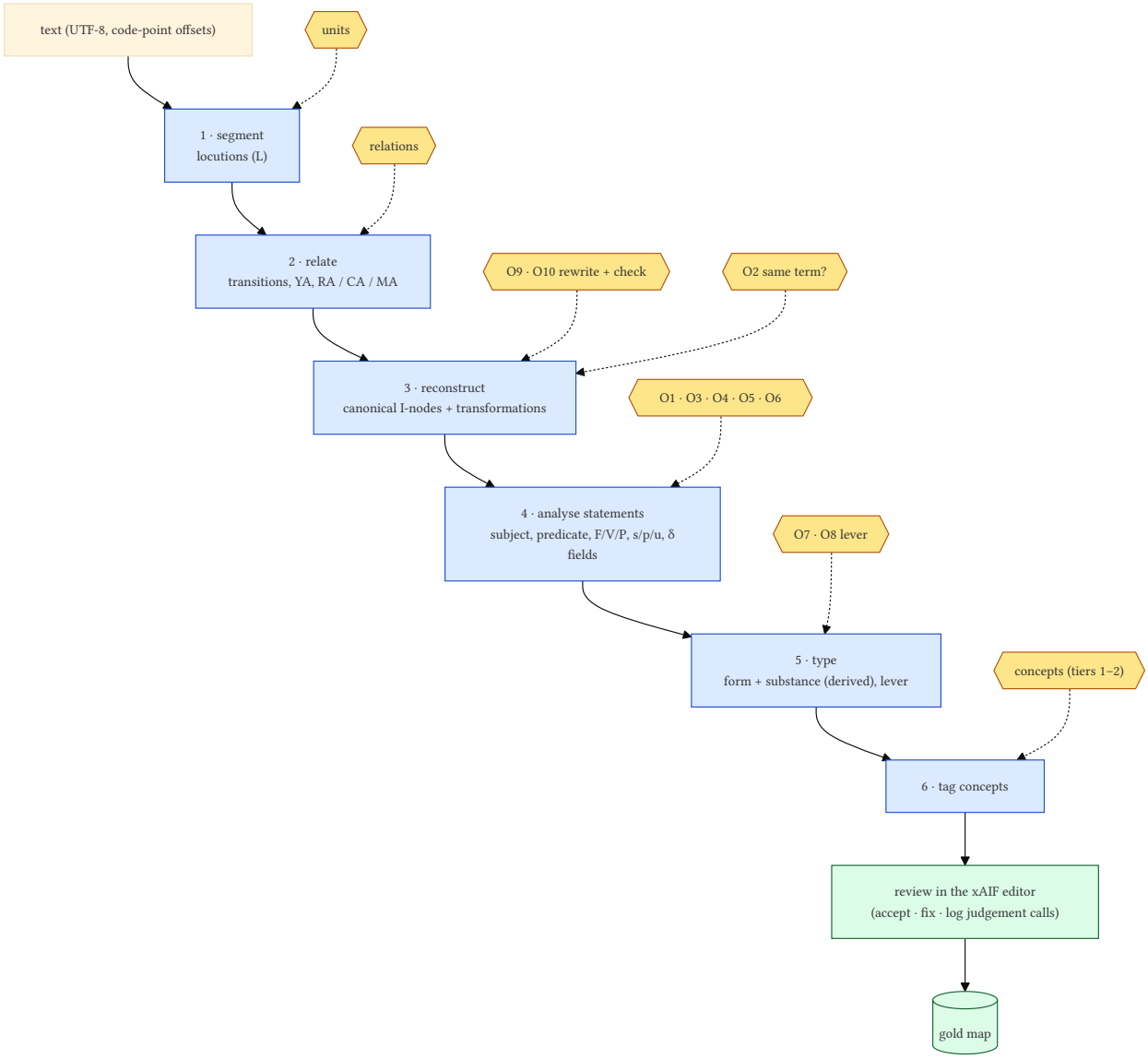
The pipeline turns a plain text into a draft argument map for a human to review. It never ships in the game, and it never has the last word.

2.1. Principle: rules, oracles and a reviewer

This principle comes from `pta-atip-algorithm.typ`. The procedure is mostly deterministic. Every judgement it can't make by rule is an **oracle**: a function that answers one small question with a probability for each possible answer. Each oracle can be answered in several ways, called **arms**, such as a rule baseline, a small model or a larger model. Arms are compared on the gold maps, one oracle at a time.

Two things are new since that design: the IAT dialogue layer, and the editor as the place where proposals are reviewed. Here is the whole pipeline:

- Rectangles are steps.
- Hexagons are oracles, each with competing arms.
- The human review at the end is not optional.



The steps follow the order a human annotator works in. Each section below says what the step decides, what can answer it, and how it's measured.

2.2. Step 1: segment the text into locutions

Decides: which spans of text are argumentative units. Each unit becomes an L-node. In the Aristotle map, "living is apparently shared with plants" is a unit; "What, then, could this be?" may not be.

Arms:

- **Rule:** split sentences and clauses with a dependency parser, and cut at discourse markers ("for", "hence", "but", "since"). Fast and transparent, but expected to struggle with long philosophical sentences.
- **Span extraction** with GLiNER2 (Section 2.5), asked for "claim" and "reason" spans.
- **A small LLM** that proposes segments, given a few examples from the gold maps.
- **Prior art:** ARG-tech's open Argument Mining Framework (oAMF) has segmenters that already output xAIF. It's worth running as an arm, since it uses our format.

Measured by: boundary overlap with the gold locutions.

> **Explainer: dependency parsing.** A parser (spaCy here) labels each word with its grammatical role and the word it depends on: "**living** (subject) **is shared** (root) **with plants** (modifier)". From that tree, a rule can find a clause's subject and predicate, or where a clause starts. The parser is small (tens of MB), fast on a CPU, and deterministic, so it is the default baseline. It was trained on news and web text, so expect it to fail on 17th-century syntax.

> **Explainer: argument mining.** Argument mining is the research field that does steps 1–2 automatically: find argumentative units, then classify the relations between them (support, attack). On edited prose, current methods find units fairly well and relations much less well, especially implicit ones. That's one reason our pipeline only *proposes*.

2.3. Step 2: relate the units

Decides: the dialogue skeleton.

- Which locution responds to which (the TAs).
- For each pair, whether one argues for, disagrees with or restates the other. That is the YA's force, and it decides between RA, CA and MA.
- Which statements are reported speech, such as Aquinas quoting Augustine.
- Which spans assert two things, so that one locution anchors two I-nodes.
- Where an unstated premise is needed.

Arms:

- **Rule:** markers decide direction and force. "For X" means X argues for the previous unit; "but" means disagreement. Reporting verbs ("says", "thinks") mark reported speech.
- **NLI** over candidate pairs: "does A support B?" phrased as an entailment test.
- **LLM** proposals for the whole passage, checked by NLI.

Measured by: precision and recall of relations against the gold maps, ignoring reconstruction.

> **Explainer: NLI: natural language inference.** An NLI model reads a **premise** and a **hypothesis** and says whether the premise **entails** the hypothesis, **contradicts** it, or neither. Example: "Every bodily part has a function" entails "The eye has a function".
>
> NLI models are small (DeBERTa-v3 xsmall is about 70 MB) and run on a CPU. They can also do **zero-shot classification**: to ask "is this lever CAUSE?", write the lever out as a hypothesis and see how strongly the argument entails it.
>
> We use NLI in four places: relations (step 2), rewrite checks (O10), ɣ's i/d substance (O7) and lever choice (O8).

2.4. Step 3: reconstruct canonical statements

Decides: each I-node's wording. This is ATIP's step 2, recorded as transformations on the YA (anaphora, passivization, same term…). This step is where **form** is really decided. The editor compares terms literally, so rewording "she" as "the minister" is what makes an argument α. It's the hardest step, and the one that has produced most of the judgement calls in `judgement-calls.md`: exclusives, kinds as universals, embedded assertions.

Oracles:

- **O2, same term?** Do two phrases refer to the same thing? Arms: lemma match after coreference resolution, embedding similarity, or a cross-encoder.
- **O9, rewrite proposer.** A generative model proposes rewordings that make a shared term visible, with their transformation tags. This is the only oracle that outputs text instead of a choice from a short list.
- **O10, rewrite check.** NLI in both directions. A rewrite is admissible if it says the same as the original *and* keeps the statement type (F, V or P).

Measured by:

- how often O9's candidates include the reviewer's rewording;
- how often O10 agrees with the reviewer's accept or reject.

> **Explainer: embeddings and cosine similarity.** An embedding model (MiniLM, BGE-small or e5-small, 20–130 MB) turns a phrase into a list of a few hundred numbers, a vector. The vectors are arranged so that phrases with similar meanings point in similar directions. **Cosine similarity** is the cosine of the angle between two vectors: 1 means "same direction", near 0 means "unrelated". Embeddings are cheap, because each phrase is encoded once and comparing two is a dot product. A model used this way is called a **bi-encoder**.

> **Explainer: cross-encoders.** A **cross-encoder** reads both phrases together and scores the pair. It is slower than embeddings, because nothing can be computed ahead of time, but it usually judges "do these mean the same?" better. A common setup: embeddings pick a shortlist, and the cross-encoder decides.

> **Explainer: small generative LLMs, in-context learning, quantization.** A generative model writes text. With **in-context learning** (also called few-shot), we put 3–8 solved examples from the gold maps in the prompt and ask the model to do the same for a new passage. No training is needed.
>
> To run on a CPU, we use small models (0.5–1.5 billion parameters), **quantized** to 4 bits: each weight is stored in 4 bits instead of 16. That cuts size and time by about 4×, with a small loss in quality. The usual way to run these is the GGUF format with llama.cpp. Even so, expect a few seconds per call on a CPU: fine offline, too slow in the game.

> **Decision needed: Local models only, or cloud LLMs for authoring too?** A large hosted model would propose much better reconstructions. That matters, because step 3 is the bottleneck. But it would send book excerpts to a third party, and the labs so far assume local models. If cloud models are acceptable for *offline proposals* (never shipped, always reviewed), one could be an arm among the others.

2.5. Step 4: analyse each statement

Decides: each I-node's fields:

- subject and predicate (O1);
- statement type: F, V or P (O3);
- subject scope: s, p or u (O4);

- whether the statement is second-order, meaning it is about a whole proposition, and if so its acceptability, A or U (O5);
- the perspective of a source: I, II or III (O6).

Arms: rules first, then small classifiers or GLiNER2. The rules:

- modals and imperatives → P; evaluative words → V;
- determiners and quantifiers → scope;
- reporting verbs → second-order;
- the speaker's role in the dialogue → perspective.

The judgement-call log already has rules a model would miss: "a claim about a kind is universal", and "F if it can be checked by looking, V if it applies a standard".

> **Explainer: GLiNER2: schema-driven extraction.** GLiNER-family models are small encoders that take their labels *at run time*. You can ask one to "find spans of type `claim`, `reason`", or to "classify this as `fact`, `value`, `policy`". New labels need no training, and the model runs on a CPU. It's already planned for the reading-notes lab (tier 2), so one integration serves both labs.

2.6. Step 5: type each argument

Decides: the PTA code. Most of this is **already code**: the editor's `deriveCode` gets the form and substance from the canonical statements by rule (`README.md` § Arguments). Two oracles remain:

- **O7** (ɣ only): is the relation between the subjects the same as the relation between the predicates (i), or not (d)? Answered by NLI.
- **O8, the lever:** usually 2–6 levers fit the form and substance. Each one is written out as a sentence and scored by NLI. Cue words ("is a SIGN of", "just as") settle it when present.

The judgement-call log adds a hard rule: `novel` is a sign the reconstruction went wrong. If the pipeline arrives at `novel`, it should send the argument back to step 3, not accept it.

Measured by: top-1 and top-3 accuracy of the code against gold, and calibration (Section 2.8).

2.7. Step 6: tag concepts

Decides: what each argument is about. These are the `concepts` on the RA, such as `faith`, `science` or `theology/christian`. This is the reading-notes lab's job, run over argument spans instead of player notes:

- tier 0: authored ranges;
- tier 1: embedding similarity to concept descriptions;
- tier 2: GLiNER2.

The same concept ids connect the argument maps to the player's notes and to the concept graph the debate uses (Section 3.2).

2.8. How the pipeline is measured

> **Explainer: stage-wise evaluation.** To find which step is failing, run the pipeline with the *gold* answers substituted for every step before the one under test. Its errors are then its own, not inherited from earlier steps. For example, to score lever choice (step 5), feed it the gold canonical statements. To score reconstruction (step 3), feed it the gold segments and relations.

- **Split by writer.** Each writer in turn is held out. The rest form the dev set, which sets thresholds and supplies few-shot examples. No writer helps classify their own text.
- **Exemplars first.** Look at the worst cases for each step before computing any statistics, as in hex-lab and note-lab.
- **Calibration.** When an arm says 80%, is it right about 80% of the time? This matters because the editor will show confidences, and reviewers need to be able to trust them.

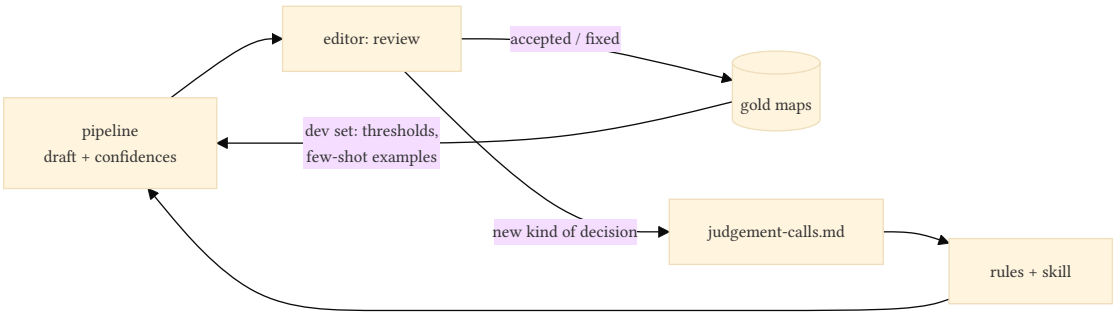
- **A human ceiling.** A second annotator on a sample shows how much people disagree. No arm should be expected to beat that.

> **Explainer: top-k and calibration. Top-1** means the arm's first choice is right. **Top-3** means the right answer is among its first three, which is useful when a reviewer picks from a short list. A model is **well calibrated** if its confidence matches its hit rate. Badly calibrated models are confidently wrong, which wastes a reviewer's attention.

2.9. How the pipeline meets the editor: suggest, then review

The labelling methods doc (§6) sets the order: first label by hand, then let models suggest, then review. Concretely:

1. The pipeline writes a **draft xAIF file**. Every proposed node and edge has a confidence and runner-ups.
2. The editor shows proposals differently from reviewed nodes (for example, dashed). The reviewer accepts, changes or deletes each one, and sees the runner-ups in the inspector.
3. Every change the reviewer makes is a labelled example. It's also a judgement call if it's a new kind of decision.
4. Over time, proposals that are always accepted become rules or defaults.



2.10. Implementation choices

- **Language.** The pipeline stays in **Python**, where the models and tools are, and where note-lab already exists. The deterministic core (derivation, lint, type table) lives in TypeScript in the editor today. The game will need a Rust version (Section 3.2).
- **One source of truth for the tables.** The PTA type table moves out of code into one JSON file, read by Python, TypeScript and Rust (`pta-lab-plan.md`, step 7).
- **A cross-language test suite.** `ra-inputs.json` already lists every way into an RA, with the code each must derive. Each implementation (TS, Python, Rust) must pass it. That keeps three copies of the rules from drifting apart.
- **Runtime for models.** ONNX Runtime in the lab and `ort` in Rust, so latency measured in the lab carries over to the game. llama.cpp for the generative arm, in the lab only.

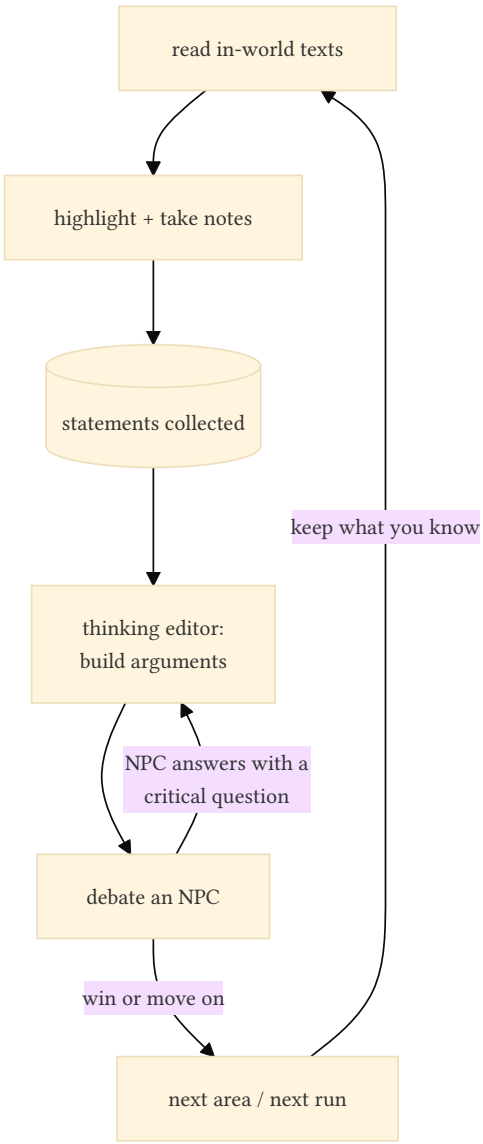
> **Explainer: ONNX and ort.** ONNX is a portable file format for trained models. ONNX Runtime runs it on many platforms, and `ort` is its Rust binding. Exporting a model to ONNX means the game can run exactly the model the lab measured, without Python.

> **Decision needed: INCEpTION or the xAIF editor as the annotation tool?** `pta-lab-plan.md` assumes INCEpTION, with the PTA fields on a Connector layer. Since then, all the real annotation has happened in the xAIF editor. It has the dialogue layer, derives types, and lints.

> Proposal: the xAIF editor becomes the gold-data tool for argument maps. INCEpTION stays only if concept or reading-notes labelling needs it. The PTA lab plan's converter (step 5) and website comparison (step 6) would then read xAIF.

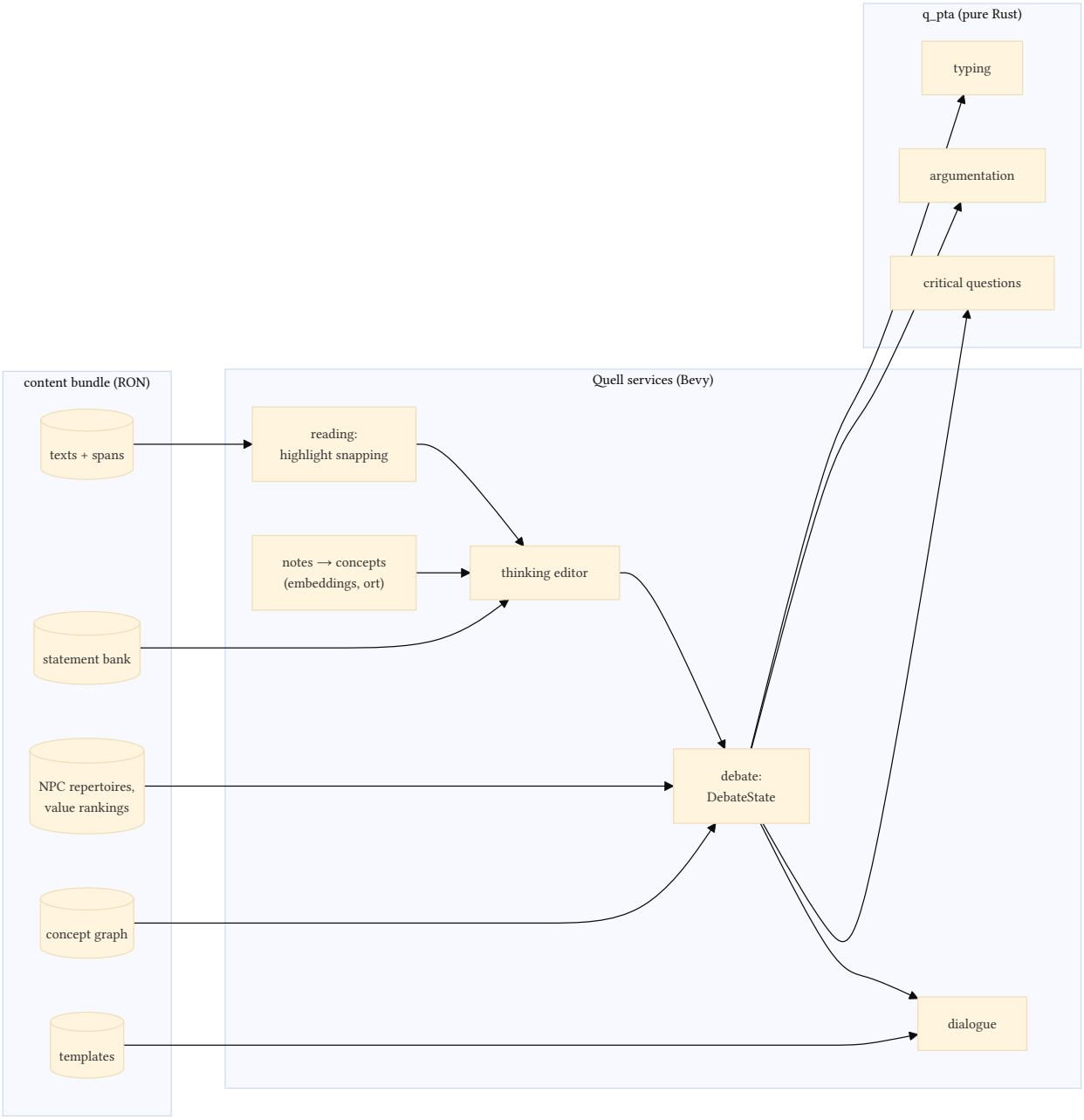
3. Integration into Quell

3.1. The player's loop



- **Reading.** The player reads a text. Highlights **snap to authored statement spans** (the map's L-nodes), so a highlight always means a known statement. Notes stay free text.
- **Collecting.** A highlighted statement goes into the player's bank, with everything the map knows about it: canonical form, type, scope and concepts.
- **Thinking.** In the graph editor, the player links a premise to a conclusion, picks a lever from the kinds they've unlocked, and can attach a lever statement. This turns ATIP into a game mechanic: the player does the typing, and that's the learning.
- **Debating.** The player submits arguments, and the NPC answers.
- **Carrying over.** A new run keeps some highlights, texts and established conclusions, which opens new dialogue paths.

3.2. Runtime architecture



3.2.1. A pure core crate

The rules that type and judge arguments have no reason to depend on Bevy. A small library crate (working name `lib/q_pta`) would hold:

- the PTA type table, loaded from the shared JSON;
- `derive_code` and `lever_options`: a port of the editor's TypeScript, checked against `ra-inputs.json`;
- the argumentation semantics: attacks, the grounded extension, and defeat under value rankings;
- the critical questions for each lever family.

Because the crate is pure, it can be tested without an `App`, and lab tools could call it later. Per `AGENTS.md`, a new crate also needs its own skill and an entry in the `q-os` ecosystem map.

```rust
/// What the player built: one connector between collected statements.
pub struct Connector {
    pub premise: StatementId,
    pub conclusion: StatementId,
    pub lever: Option<LeverKind>, // chosen by the player
    pub lever_statement: Option<StatementId>,
}

/// The rule-derived type; None if the statements share both terms.
pub fn derive_code(bank: &StatementBank, c: &Connector) -> Option<PtaCode>;

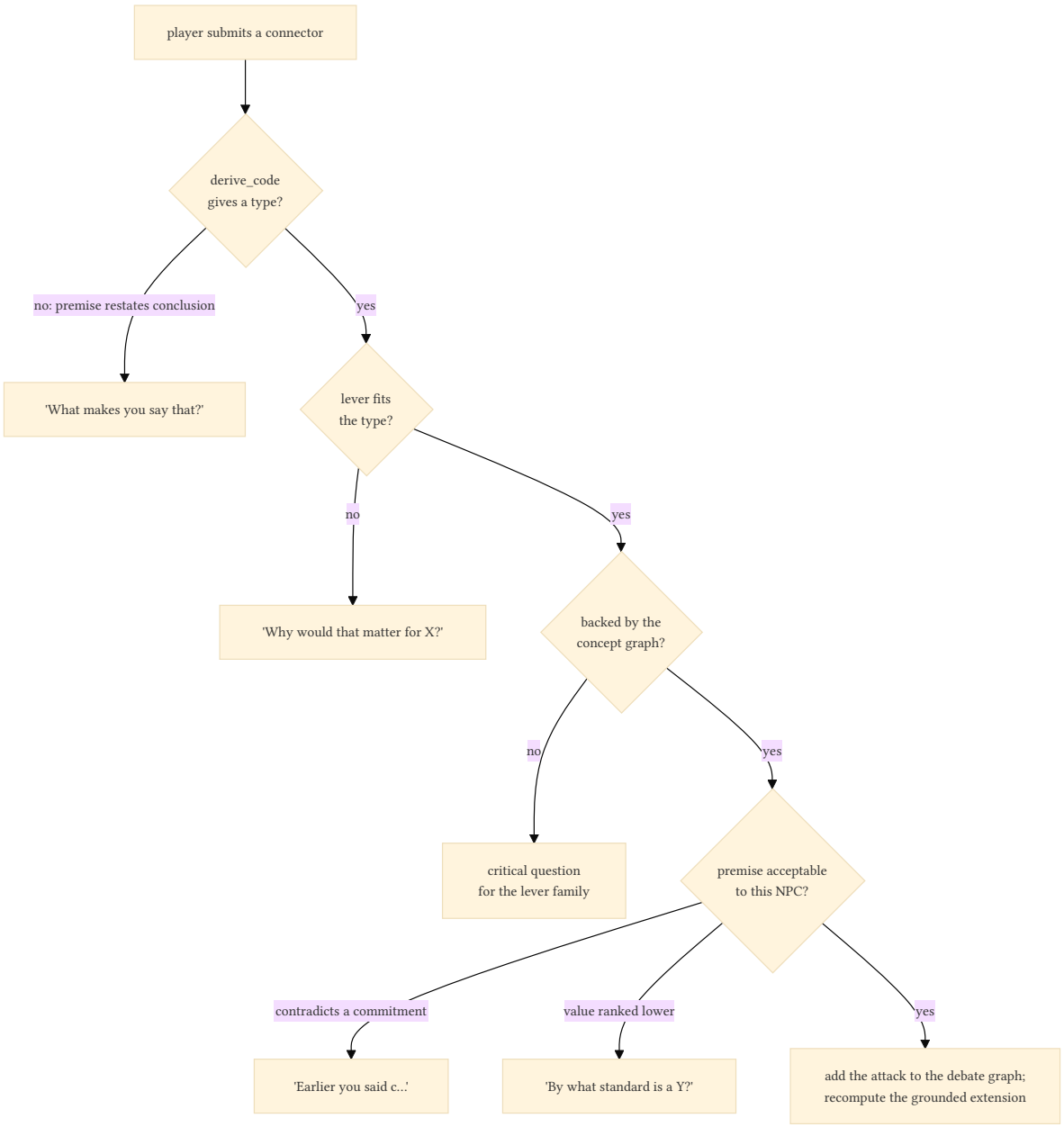
/// Whether the game's knowledge backs the connector's lever.
pub fn backing(bank: &StatementBank, graph: &ConceptGraph, c: &Connector) -> Backing;
```

3.2.2. Judging what the player builds

The player can combine statements in ways nobody authored. There are two extremes:

- **Lock and key:** only authored connections count, as in *Ace Attorney*. It's simple, but it isn't debate.
- **Rules over typed metadata** (proposed). Each statement carries its analysis and concepts, and the concept graph has typed edges (`causes`, `part-of`, `analogous`, `conflicts`…). A connector is **backed** when that knowledge supports its lever. For example, `β uu Cpo` is backed only if the graph says the conclusion's subject is composed of the premise's subject.

  An unbacked connector isn't wrong. The NPC answers with that lever's critical question, and the player can defend by attaching a lever statement. That is the PTA's recursion, made into a player action.



The response categories are the table in `capna-based-flow.md`: *(stage, reason)* pairs, each with a template. The template's slots are filled from the argument: a, X, b, Y, the source, the commitment.

> **Decision needed: How do we stop brute force?** With a small bank, a player can try every combination. Options:
>
> - a cost per submission (NPC patience, a limited number of turns);
> - responses that reveal only the *category* of failure, not the fix (the templates already work this way);
> - confirming in batches, as *Return of the Obra Dinn* does.
>
> Probably a mix, settled by playtesting.

3.2.3. Where models still run in the game

Only one place needs a model at runtime: **interpreting notes**. A note is free text, and the game wants concept ids out of it, to gate dialogue and suggest connections. That is the reading-notes lab's tier 1: an embedding model run via `ort` (20–130 MB, milliseconds per note on a CPU), measured under the lab's throttle profiles. Everything else is rules and tables: typing, judging, and choosing a response.

> **Explainer: why not an LLM in the game?.** A 0.5B model quantized to 4 bits is about 400 MB, and takes seconds per reply on a CPU while competing with the game for cores. Its output can't be checked before the player sees it, and it can make up philosophy. Templates do the same job in a debate. The hard part is the *choice* of what to say, and that comes from the rules. The wording is authored.

3.2.4. Dialogue

The lab's `examples/*.yarn` files sketch the dialogue in Yarn: the evaluator sets variables (`$argumentIsValid`, `$foundConcept`) and the script jumps to nodes. A Bevy integration for Yarn Spinner exists (`bevy_yarnspinner`), but we haven't evaluated it yet. With it, the debate service would set Yarn variables from the evaluation and jump to the template node for the (stage, reason).

3.2.5. Assets and the exporter

The exporter, on the lab side, turns reviewed maps into Quell assets:

- **texts and locution spans**, for reading and highlight snapping;
- **the statement bank**: canonical I-nodes with their analysis and concepts;
- **NPC repertoires**: the arguments each NPC can make (taken from the map or authored), their commitments, and their value ranking;
- **the concept graph**, built up while annotating;
- **the templates**.

RON is the natural format, since the repo already saves RON assets, loaded through Bevy's asset system. Tweakable numbers (patience, thresholds, carry-over rules) are Resources, per `AGENTS.md`.

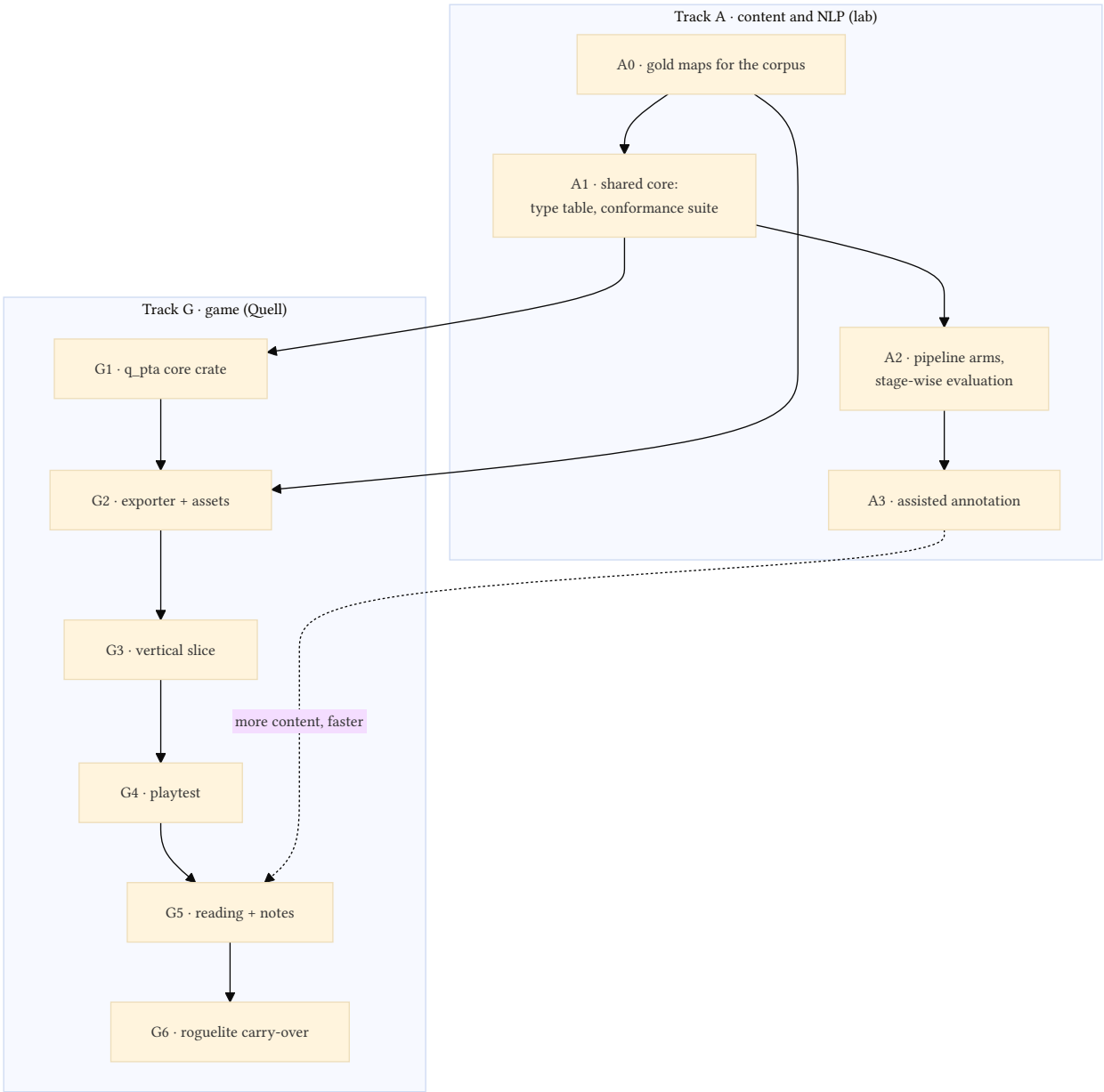
3.2.6. Fitting Quell's structure

Following `bin/quell/docs/architecture.md`:

- `services/reading/`: text display, highlight snapping, the player's bank;
- `services/notes/`: notes and their interpretation;
- `services/debate/`: `DebateState` (the shared argument graph), submission, evaluation via `q_pta`, response selection;
- `screens/debate/` and the thinking editor UI;
- a persistent **knowledge** save, for what carries between runs.

4. Phases

The plan has two tracks, which meet only at the exporter. The game track can start from the hand-made maps; it doesn't wait for the pipeline.



4.1. Track A: content and NLP

A0 · Gold maps (now). 🧑 critiques, 🤖 drafts

Draft a map for each corpus text, critique it, fix it, and log the judgement calls. Aquinas and Engels are done; Aristotle is drafted.

- *Done when:* a first batch of maps has been reviewed (say 10 of the 31), and new judgement calls have mostly stopped appearing.
- *Then:* fold the settled calls into a labelling guide. If the tool decision goes that way, this replaces §3 of the INCEpTION-era `guide.typ`.

A1 · Shared core. 🤖

The type table as JSON; `ra-inputs.json` as the conformance suite; a Python reader for xAIF with the same derivation and lint, tested against the suite.

- *Risk:* low. The TypeScript code is the reference.

A2 · Pipeline arms. 🤖 builds, 🧑 reviews exemplars
In order of expected payoff:
1. step 5 (lever, NLI) and step 4 (statement fields: rules and GLiNER2), which are small closed-set questions;
2. steps 1–2 (segmentation and relations: rules, oAMF, an LLM);
3. step 3 (reconstruction, O9/O10), the hardest.

Each arm is scored stage-wise against the gold maps.
- *Risk:* step 3 may never get good enough to save time. That's acceptable; the other steps still save clicks.
- *Downloads:* each model is a separate approval (disk, licences).

A3 · Assisted annotation. 🤖 builds, 🧑 uses
The pipeline writes draft xAIF, and the editor shows the proposals with confidences and runner-ups. Measure time per text and corrections per proposal.

4.2. Track G: the game

G1 · Core crate. 🤖
`q_pta`: the type table, `derive_code`, lever options, the grounded extension, defeat under value rankings, and critical questions. It must pass the conformance suite.

G2 · Exporter and assets. 🤖
Reviewed xAIF ⟶ a RON bundle. Start with one text.

G3 · Vertical slice. 🤖 builds, 🧑 designs and tunes
The scope agreed in the earlier discussion:
- one text with about 20 statements;
- 3–4 lever kinds;
- one NPC with a value ranking;
- a minimal thinking editor: place statements, link premise ⟶ conclusion, pick a lever;
- the response templates.

Highlighting can be faked at first, by giving the player every statement.

G4 · Playtest. 🧑
The question to answer: *is building one argument against one NPC fun?* If it isn't, nothing later will fix it.

G5 · Reading and notes. 🤖 builds, 🧑 tunes
Highlight snapping, the bank, and notes ⟶ concepts (tier 1 via `ort`), measured under the throttle profiles.

G6 · Carry-over. 🧑 designs, 🤖 builds
What persists between runs, and which dialogue paths it opens.

4.3. Deferred

- **Free-text player arguments** (the original CAPNA flow). This could come back later as "propose a new statement": the text is matched to the bank by embeddings, with "What do you mean by…?" as the fallback.
- **Fine-tuning** any model. Not until the materials are final, as in note-lab.
- **Running any generative model in the game.**

5. Risks and open questions

1. **Reconstruction is the bottleneck.** Form depends on wording, and wording is a judgement. If step 3 can't be automated well, authoring stays manual, and the amount of content is set by how fast one person can annotate.
2. **Is the PTA's level of detail right for play?** 50 types is a lot to teach. The thinking editor might show lever *families* (cause, example, analogy, authority…) and keep the full code internal.
3. **Authoring the concept graph.** Judging unauthored connectors needs typed concept edges. How many, and who writes them? The maps could suggest some: every reviewed `β Cpo` implies a `composed-of` edge.

4. **What counts as winning a value debate?** In value-based frameworks, the outcome depends on the audience. Is it satisfying to lose because an NPC simply ranks values differently? The game may need to say so explicitly ("you won't move me on liberty").
5. **Shipping real texts.** The lab never commits the excerpts. Shipping them in the game is a licensing question for each text: public-domain translations, or permission. This could limit which writers appear.
6. **Three copies of the rules** (TypeScript, Python, Rust). The conformance suite keeps them aligned. The alternative is compiling the Rust core to WASM for the editor and Python, which is more work up front.
7. **The reported-speech convention** is still open in the judgement-call log: do reported locutions sit in the reporter's TA chain? It affects the segmentation rules and the exporter.
8. **Dialogue tool.** Yarn is sketched, but not yet evaluated in Bevy.

5.1. Glossary

arm	one way to answer an oracle: a rule, or a model with a configuration
ATIP	Argument Type Identification Procedure: six steps from a text to a PTA type
CAPNA	a procedure for judging an argument: process, reasoning, language
canonical statement	an I-node reworded so that shared terms are literally shared
connector	one argument: premise, conclusion and an optional lever statement (an RA)
critical question	the standard challenge to a lever family, e.g. "is a really like b?"
gold	human-reviewed data that models are measured against
grounded extension	the arguments that survive every attack, computed by a simple loop
IAT	Inference Anchoring Theory: locutions, propositions and the links between them
lever	the relation that makes a premise support a conclusion (CAUSE, EXAMPLE…)
NLI	natural language inference: does text A entail text B?
oracle	a named judgement the rules can't make, answered with probabilities
PTA	Periodic Table of Arguments: 50 types from form, substance and lever
xAIF	ARG-tech's JSON format for IAT graphs, and our editor's file format

5.2. References

- Bench-Capon, T. (2003). Persuasion in practical argument using value-based argumentation frameworks. *Journal of Logic and Computation* 13(3).
- Dung, P. M. (1995). On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence* 77(2).
- Hinton, M. and Wagemans, J. H. M. (2023). CAPNA. In `.docs/reference/`.
- IAT guidelines and tutorials (2023). `.docs/reference/IAT_guidelines_and_tutorials-2023-10.pdf`.
- Wagemans, J. H. M. (2026). ATIP version 6 and PTA 3.1. In `.docs/reference/`.
- ARG-tech: xAIF and the open Argument Mining Framework (oAMF), https://github.com/arg-tech.
- In this repo: `pta-atip-algorithm.typ`, `pta-document-analysis.typ`, `capna-based-flow.md`, `quell-reading-notes-design.md`, `pta-lab-plan.md`, `misc/labs/xAIF/README.md`, `judgement-calls.md`.